实验摘要：

1、 用 Matlab、python 等编程语言实现题目给出的相关信号，以及信号的相关变化，绘制出图形。
2、 增强学生的离散信号和连续信号的理解。
3、 加深学生对常见信号的理解与认识，如门函数、阶跃函数等。
4、 提高学生应用编程语言解决信号问题的能力。

实验题目：

1. 利用 MATLAB 实现下列信号，并绘出图形

(1)$f_1(t)=\varepsilon(2t)$，取$t=-10\sim10$

(2)$f_2(t)=4e^{-2t}\cos(\pi t)$，取$t=0\sim10$

(3)$f_3(t)=g_2(t)+g_6(t)$，取$t=-10\sim10$

(4)$f_4(k)=\varepsilon(k+3)-\varepsilon(k-2)$

(5)$f_5(k)=5(0.4)^k\cos(1.2\pi k)$

(6)$f_6(t)=Sa(t/2)=2\sin(t/2)/t$

2. 利用 MATLAB 实现以上信号 $f_3(t)$ 的变化：

(1)$f_3(0.5t)$

(2)$f_3(2-2t)$

(3)$f_3'(2-2t)$

3．画出下式表示的图像，其中 n=200，t 在 0-4π中取 1000 个点

$$\sum_{k=1}^{n}\frac{\sin((2k-1)t)}{(2k-1)}$$

描述题目：

第一题使用编程语言根据所给信号绘制图形，1）为阶跃函数，2）3）为连续信号，4）和 5）为离散信号，根据不同给定的 t 或 k 范围来进行绘图。

第二题使用编程语言对 f3（x）进行信号变换，其中涉及到求导的相关操作。

第三题使用编程语言来对这个式子进行实现绘图，涉及求和等相关操作。

实验内容：

一、实验基本原理及步骤

1-（1）

在 t= -10~10 范围内绘制 f1 信号，主要设计阶跃函数的实现，阶跃函数是当 t<0 时输出 0，t>=0 时输出 1，利用条件判断语句即可实现。

具体实验代码如下：

```
#question 1
t_value = np.linspace(-10, 10, 1000)
def fun_1(t):
    return np.where(t > 0, 1, 0)
y = []
for i in t_value:
    print(i)
    y.append(fun_1(2 * i))
plt.plot(t_value, y)
plt.xlabel('t')
plt.ylabel('F1')
plt.title('F1')
plt.grid()
plt.show()
```

1-（2）

在 t=0~10 范围内绘制 f2 信号，主要设计到指数函数和三角函数的表示，可以调用 numpy 库也可以调用 math 库来解决。

具体实验代码如下：

```
#question 2
t = np.linspace(0, 10, 1000)
y = 4 * np.exp(-2 * t) * np.cos(np.pi * t)
plt.plot(t, y)
plt.xlabel('t')
plt.ylabel('F2')
plt.title('F2')
plt.grid()
plt.show()
```

1-（3）

需要理解 g2（t）和 g6（t）的意思，以 g2（t）为例，表示（-1，1）之间函数值为 1，其他值为 0，g6（t）同理，t 在-10~10 的范围内发生变化。

具体实验代码如下：

```
# question 3

def fun_3(t):
    return np.where((t > -1) & (t < 1), 1, 0)


def fun_4(t):
    return np.where((t > -3) & (t < 3), 1, 0)


t = np.linspace(-10, 10, 1000)
f1 = fun_3(t)
f2 = fun_4(t)
y = f1 + f2
plt.plot(t, y)
plt.xlabel('t')
plt.ylabel('F3')
plt.title('F3')
plt.grid()
```

1-（4）

第一问的阶跃函数是连续的，这一小题实际上是将连续变成离散，与第一问思路一样，在绘图过程中需要注意离散点向 x 轴做垂线，需要将垂线合理的表示出来。

具体实验代码如下：

```
#question 4
t =range(-10,10)
def fun_5(t):
    return np.where(t > -3, 1, 0)
def fun_6(t):
    return np.where(t > 2, 1, 0)
y = []
for i in t:
    y.append(fun_5(i)-fun_6(i))
# 绘制散点图
plt.scatter(t, y)
# 添加垂直线
for i in range(len(t)):
    plt.vlines(t[i], 0, y[i], colors='gray', linestyles='dotted')
plt.xlabel('t')
plt.ylabel('F4')
plt.title('F4')
plt.grid()
plt.show()
```

1-（5）

这个小题和第二问相似，第二问是连续的，这一问是离散的点，解决方法和第二问类似，同样需要注意，离散点向 x 轴做垂线，需要将垂线合理的表示出来。

具体实验代码如下：

```
#question 5
t = range(-10, 10)
y = []
for i in t:
    y.append(5.0*(0.4)**i*np.cos(1.2*np.pi*i))
# 绘制散点图
plt.scatter(t, y)
# 添加垂直线
for i in range(len(t)):
    plt.vlines(t[i], 0, y[i], colors='gray', linestyles='dotted')
plt.xlabel('t')
plt.ylabel('F5')
plt.title('F5')
plt.grid()
plt.show()
```

1-（6）

本题主要是连续信号，需要用将 sin 函数调用出来，即可实现信号。

具体实验代码如下：

```
#question 6
t = np.linspace(-10, 10, 1000)
y = 2* np.sin(t/2)/t
plt.plot(t, y)
plt.xlabel('t')
plt.ylabel('F6')
plt.title('F6')
plt.grid()
plt.show()
```

2-（1）

在 f3 函数基础上对 t 乘以 0.5，代码容易实现，即在每次调用函数的时候对实参进行对应操作，实际上图像变换是在 1-3）图像的基础上对图像拉伸 2 倍。

具体实验代码如下：

```
#question 7
def fun_3(t):
    return np.where((t > -1) & (t < 1), 1, 0)


def fun_4(t):
    return np.where((t > -3) & (t < 3), 1, 0)
f3 = fun_3(0.5*t)
f4 = fun_4(0.5*t)
y1 = f3+f4
plt.plot(t, y1)
plt.xlabel('t')
plt.ylabel('y')
plt.title('2-(1)')
plt.grid()
plt.show()
```

2-（2）

在 f3 函数基础上先对函数图像向左平移 2 个单位，再对 y 轴进行翻转，然后图像再压缩 1/2，得到新的图像，代码实现与 2-（1）类似，在传入实参的时候进行相关操作即可。

具体实验代码如下：

```
def fun_3(t):
    return np.where((t > -1) & (t < 1), 1, 0)
```

```
def fun_4(t):
    return np.where((t > -3) & (t < 3), 1, 0)
f5 = fun_3(2-2*t)
f6 = fun_4(2-2*t)
y2 = f5+f6
plt.plot(t, y2)
plt.xlabel('t')
plt.ylabel('y')
plt.title('2-(2)')
plt.grid()
plt.show()
```

2-（3）

在 2-（2）基础上需要实现导数的功能，通过查阅资料发现可以调用 python 的 numpy 库的函数 differ（）函数来通过差分实现导数功能。

具体实验代码如下：

```
# 计算离散数据点的导数
dt = t[0] - t[1]  # 计算 x 的间距
dy = np.diff(y2) / dt  # 使用中心差分法计算 y 的导数

dy = np.append(dy, [dy[-1]])  # 补充末尾导数值，使长度与原始数据相同

plt.plot(t, dy)
plt.xlabel('t')
plt.ylabel('dy')
plt.title('2-(3)')
plt.grid()
plt.show()
```

3

主要涉及到 k 和 t 的遍历，可以用两个嵌套的循环来进行求解，得到 y 值，进而绘图得到图形。

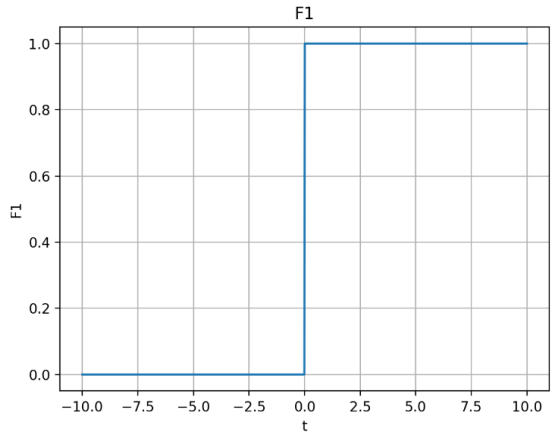
具体实验代码如下：

```
t = np.linspace(0, np.pi*4, 1001)
y = []
for i in t:
    x = 0
    for j in range(1, 201):
        x += np.sin((2*j-1)*i) / (2*j-1)
    y.append(x)
plt.plot(t, y)
plt.xlabel('t')
plt.ylabel('y')
plt.title('3')
plt.grid()
plt.show()
```

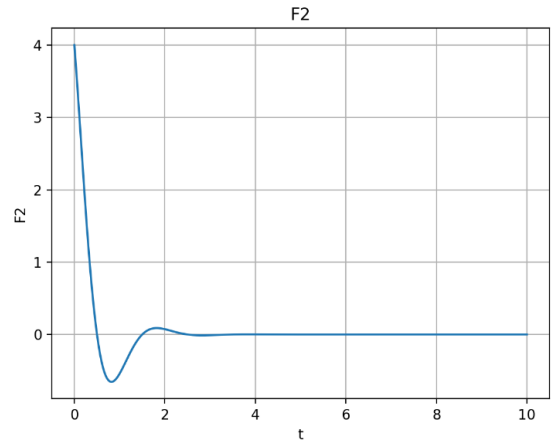
二、实验结果

实验最终绘制的图形如下一一所示：

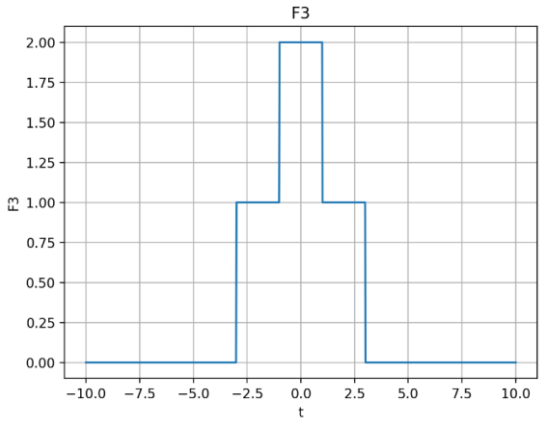
1-（1）


F1
1.0
0.8
0.6
0.4
0.2
0.0
F1
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
t

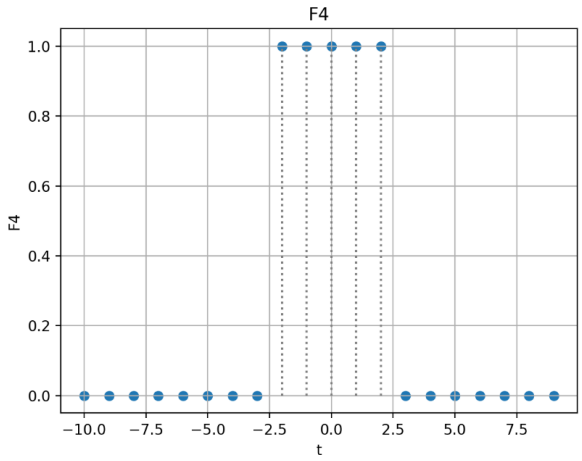
1-（2）


F2
4
3
2
1
0
F2
0
2
4
6
8
10
t

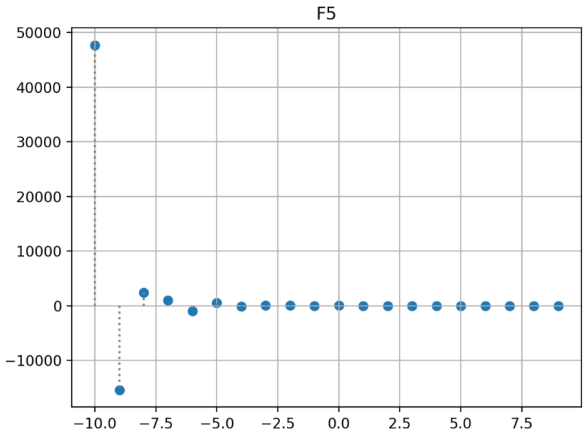
1-（3）


F3
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
F3
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
t

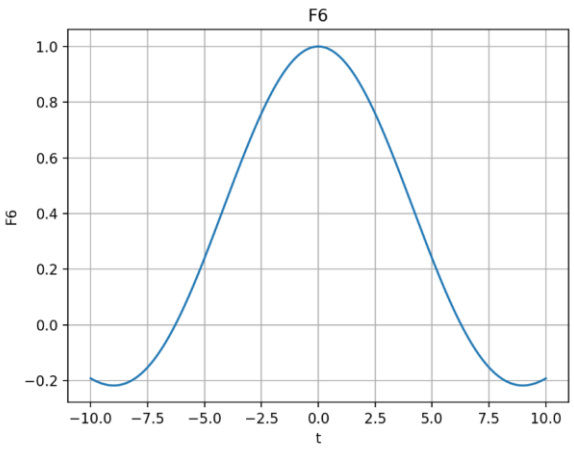
1-（4）


F4
1.0
0.8
0.6
0.4
0.2
0.0
F4
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
t

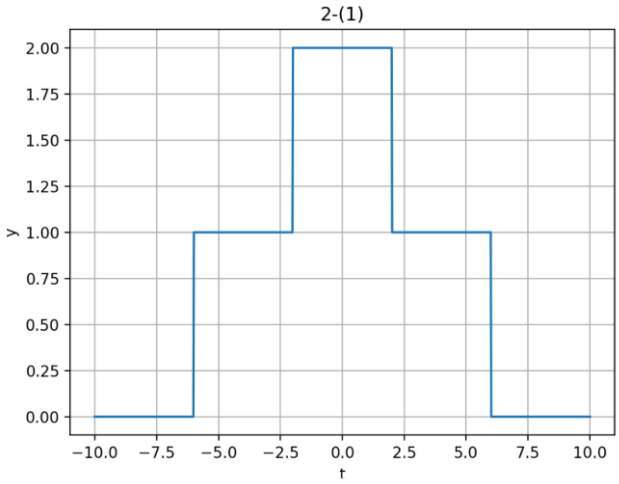
1-（5）


F5
50000
40000
30000
20000
10000
0
−10000
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5

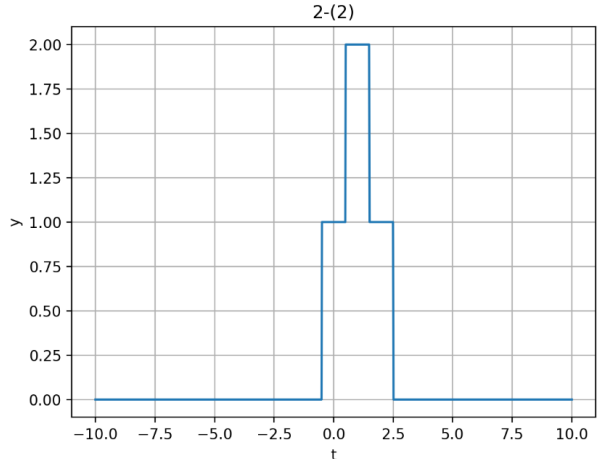
1-（6）


F6
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
F6
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
t

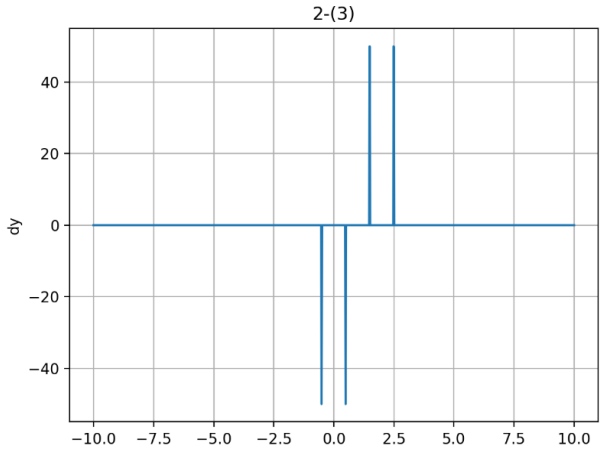
2-（1）



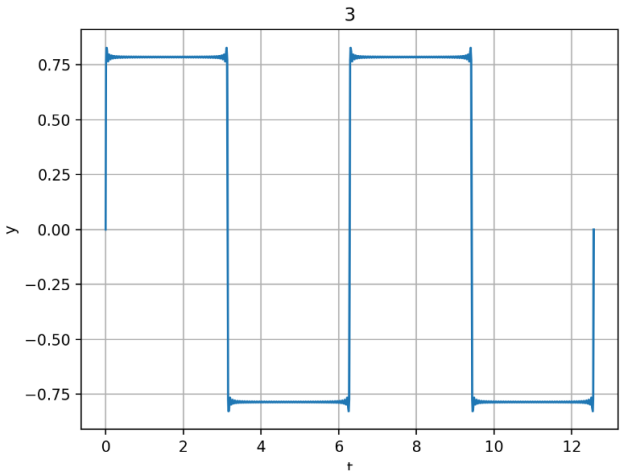
2-（2）



2-（3）



3



三、实验结果分析

根据学习观察所要绘制的图形，能够大致清楚最终需要得到的函数图形，跟代码图形比对即可确定绘制图像的结果是否正确。

其中第三题实际上是傅里叶变换的可视化，多个正弦波的叠加可以绘制出方波的形状，当 n 足够大，方波形状会更规则一点。

实验总结：

心得：

加深了对连续信号和离散信号图形的理解，进一步掌握了用编程语言来绘图的相关能力，对于常见的信号，如门函数、阶跃函数等也有了新的认识。

碰到的问题和解决方法：

1、 g2（t）这个函数符号不认识，通过去询问同学了解到这个函数。

2、 对 python 相关的数学符号，如指数函数、三角函数的调用还不够熟练，对 python 绘图的相关库操作掌握的也不是很熟练。解决方法：网络查找相关资料，主要针对 matplotlib 和 numpy 库的相关功能的搜索。

3、 对于第二题的第三问导数相关函数图像的绘制不是很理解。解决方法：网上搜索相关问题，得到差分求解导数的方法，并应用到实验中。

参考文献

1、 numpy 库的相关使用：

https://blog.csdn.net/m0_74344139/article/details/134842295

2、 matplotilb 库的相关使用：

https://www.matplotlib.net/stable/index.html

3、 python 绘制离散点的微分：

https://blog.csdn.net/goodCodeVsBadBs/article/details/108270333